

Haskell 復習ノート

型システム・値コンストラクタ・レコード型・多相型

Hiroki Funashima

Thursday, May 21, 2026



目次

1	このノートの目的	5
2	Haskell における型	5
2.1	式には型がある	5
2.2	型を集合のように考える	5
2.3	静的型付け	6
3	型推論	6
4	関数の型	7
4.1	関数も値である	7
4.2	カーリー化	7
5	代数的データ型	8
5.1	<code>data</code> 宣言	8
5.2	型レベルと値レベル	8
6	型コンストラクタと値コンストラクタ	8
6.1	2 種類のコンストラクタ	8
6.2	型コンストラクタ	9
6.3	値コンストラクタ	9
7	値コンストラクタは関数である	10
8	値コンストラクタとパターンマッチ	10
9	直積型	11
9.1	複数の情報を同時に持つ型	11
10	直和型	11
10.1	複数の可能性のどれかを表す型	11
11	Maybe	12
11.1	失敗可能性を型で表す	12
11.2	Maybe の構造	13
12	Either	13
12.1	理由つきの分岐	13
13	リスト型	14

14	レコード型	15
14.1	通常データ型の弱点	15
14.2	レコード構文	15
14.3	レコード型の値を作る	16
14.4	レコード型から値を取り出す	16
14.5	レコード型とパターンマッチ	16
14.6	レコード更新構文	17
14.7	レコード型は名前付き直積型	17
15	レコード型の注意点	18
15.1	フィールド名は関数名でもある	18
15.2	部分的なアクセサに注意	19
16	型変数	19
16.1	型変数とは何か	19
16.2	型変数は使われる場所で具体化される	20
17	多相型	20
17.1	多相型とは何か	20
17.2	型変数の名前	21
18	型変数を持つデータ型	21
19	制約なし多相と制約付き多相	22
19.1	制約なし多相	22
19.2	制約付き多相	22
20	多相型の重要な性質	23
20.1	多相性は自由度を制限する	23
20.2	型から実装を推測できる	23
21	同じ型変数が複数回出る意味	24
22	レコード型と多相型の組み合わせ	25
22.1	多相的な状態型	25
22.2	型変数は設計上の自由度を表す	26
23	type と newtype	26
23.1	type は別名	26
23.2	newtype は新しい型を作る	26
24	型クラス制約とレコード型	27

25	型が設計を表す	28
26	説明順序の提案	28
27	まとめ	29

1 このノートの目的

このノートでは、Haskell の基本的な型システム、代数的データ型、値コンストラクタ、レコード型、型変数、多相型について整理する。

単なる文法事項としてではなく、次の観点を重視する。

要点

Haskell における型は、単なるデータの分類ではなく、プログラムの意味構造や設計意図を表すものである。

特に、Haskell では型定義を見るだけで、あるデータがどのような構造を持ち、どのような場合分けを持ち、どのように使われるべきかがかなり明確になる。

2 Haskell における型

2.1 式には型がある

Haskell では、ほぼすべての式に型がある。

```
42      :: Int
True     :: Bool
"hello"  :: String
[1, 2, 3] :: [Int]
```

記号 `::` は「この式はこの型を持つ」という意味である。

```
42 :: Int
```

これは、

42 という値は Int 型の値である

という意味である。

2.2 型を集合のように考える

初学者向けには、型を「値が属する集合」のように考えるとわかりやすい。

例えば、Bool 型には次の 2 つの代表的な値がある。

```
True  :: Bool
False :: Bool
```

したがって、教育的にはまず、

型とは、値の種類を表すものである

と説明してよい。

ただし、Haskell には遅延評価、undefined、非停止計算などがあるため、厳密には「型＝集合」と完全に同一視するのはやや危険である。より正確には、型は「値や計算の振る舞いを制約する分類」と考えるのがよい。

2.3 静的型付け

Haskell は静的型付け言語である。

つまり、プログラムを実行する前に型の整合性を検査する。

```
1 + 2
```

は問題ないが、

```
1 + True
```

は型エラーになる。

+ は数値に対する演算であり、True のような Bool 型の値を数値として足すことはできない。

要点

静的型付けにより、多くの誤りを実行前に検出できる。

3 型推論

Haskell では、多くの場合、型注釈を書かなくてもコンパイラが型を推論する。

```
double x = x * 2
```

この関数の型は、おおよそ次のように推論される。

```
double :: Num a => a -> a
```

これは、

double は、Num 型クラスに属する任意の型 a について、a 型の値を受け取り、a 型の値を返す関数である

という意味である。

GHCi では、次のように型を確認できる。

```
:type double
```

4 関数の型

4.1 関数も値である

Haskell では、関数も値である。

```
not :: Bool -> Bool
```

これは、

Bool 型の値を受け取り、Bool 型の値を返す関数

である。

また、

```
length :: [a] -> Int
```

は、

任意の型 a のリストを受け取り、その長さを Int として返す関数

である。

4.2 カリー化

Haskell では、複数引数関数は本質的には「1 引数関数を返す関数」として扱われる。

例えば、

```
(+) :: Num a => a -> a -> a
```

は、次のように読むこともできる。

```
(+) :: Num a => a -> (a -> a)
```

つまり、

まず a 型の値を 1 つ受け取り、次に a -> a 型の関数を返す

ということである。

例えば、

```
addOne :: Num a => a -> a  
addOne = (+) 1
```

のように、部分適用によって新しい関数を作れる。

5 代数的データ型

5.1 data 宣言

Haskell の型システムを理解する上で中心になるのが、代数的データ型である。
典型例として、Bool は次のように考えられる。

```
data Bool = False | True
```

ここで、

- Bool は型名である。
- False と True は値コンストラクタである。

したがって、

```
False :: Bool  
True  :: Bool
```

である。

5.2 型レベルと値レベル

重要なのは、Bool と True / False が同じ種類のものではないという点である。

```
Bool
```

は型レベルの名前であり、

```
True  
False
```

は値レベルの名前である。

要点

型は値の種類を表し、値コンストラクタはその型に属する値を作る。

6 型コンストラクタと値コンストラクタ

6.1 2 種類のコンストラクタ

Haskell では「コンストラクタ」という言葉が 2 つの意味で出てくる。

1. 型コンストラクタ
2. 値コンストラクタ

この 2 つは区別する必要がある。

6.2 型コンストラクタ

型コンストラクタは、型を作るものである。

例えば、次のようなものが型コンストラクタである。

```
Maybe
Either
[]
IO
```

Maybe は単独では具体的な値の型ではない。

```
Maybe Int
Maybe String
Maybe Bool
```

のように、型引数を与えることで具体的な型になる。

kind の記法を使うなら、概念的には次のように表せる。

```
Maybe :: Type -> Type
Either  :: Type -> Type -> Type
```

つまり、Maybe は型を 1 つ受け取って型を作り、Either は型を 2 つ受け取って型を作る。

6.3 値コンストラクタ

値コンストラクタは、値を作るものである。

例えば、Maybe は次のように定義できる。

```
data Maybe a
  = Nothing
  | Just a
```

このとき、

- Maybe は型コンストラクタである。
- Nothing と Just は値コンストラクタである。

値コンストラクタの型は次のようになる。

```
Nothing :: Maybe a
Just     :: a -> Maybe a
```

Nothing は引数なしで Maybe a 型の値を作る。

Just は a 型の値を受け取り、Maybe a 型の値を作る。

```
Just 3      :: Maybe Int
Just "hello" :: Maybe String
Nothing     :: Maybe a
```

7 値コンストラクタは関数である

次の型を考える。

```
data Point = Point Double Double
```

このとき、右辺の Point は値コンストラクタであり、次の型を持つ。

```
Point :: Double -> Double -> Point
```

したがって、

```
p :: Point
p = Point 1.0 2.0
```

と書ける。

ここで、左辺の Point と右辺の Point は名前が同じだが、役割が異なる。

```
data Point = Point Double Double
--   ^型名   ^値コンストラクタ
```

必ず同じ名前にする必要はない。

```
data Point = MkPoint Double Double
```

と書けば、

```
MkPoint :: Double -> Double -> Point
```

となる。

8 値コンストラクタとパターンマッチ

値コンストラクタは、値を作るときだけでなく、分解するときにも使われる。

```
data Point = Point Double Double

distanceFromOrigin :: Point -> Double
distanceFromOrigin (Point x y) =
    sqrt (x * x + y * y)
```

ここで、

```
Point x y
```

は、Point コンストラクタで作られた値を分解し、中の 2 つの Double を x と y として取り出すパターンである。

要点

値コンストラクタは、値の構築にも、値の分解にも関わる。

論理的に言えば、

- コンストラクタは導入規則
- パターンマッチは除去規則

のように見ることもできる。

9 直積型

9.1 複数の情報を同時に持つ型

次の型を考える。

```
data Person = Person String Int
```

これは、

```
Person :: String -> Int -> Person
```

という値コンストラクタを定義する。

```
taro :: Person  
taro = Person "Taro" 20
```

この型は、

名前 String と年齢 Int を同時に持つ値

を表している。

数学的には、おおよそ

$$\text{Person} \simeq \text{String} \times \text{Int}$$

と考えられる。

このように、複数の情報を同時に持つ型は直積型として理解できる。

10 直和型

10.1 複数の可能性のどれかを表す型

次の型を考える。

```
data Shape  
  = Circle Double  
  | Rectangle Double Double
```

これは、

```
Circle    :: Double -> Shape  
Rectangle :: Double -> Double -> Shape
```

という値コンストラクタを持つ。

つまり、Shape 型の値は、

- 半径を持つ円
- 幅と高さを持つ長方形

のどちらかである。

```
s1 :: Shape
s1 = Circle 3.0

s2 :: Shape
s2 = Rectangle 4.0 5.0
```

面積を計算する関数は次のように書ける。

```
area :: Shape -> Double
area (Circle r) =
    pi * r * r
area (Rectangle w h) =
    w * h
```

数学的には、おおよそ

$$\text{Shape} \simeq \text{Double} + (\text{Double} \times \text{Double})$$

と考えられる。

要点

data 宣言の | は「または」を表す。複数の値コンストラクタを持つ型は、場合分けを型に埋め込んでいる。

11 Maybe

11.1 失敗可能性を型で表す

Maybe は、値が存在しない可能性を表す型である。

```
data Maybe a
    = Nothing
    | Just a
```

これは、

a 型の値があるかもしれないし、ないかもしれない

という意味である。

例えば、安全な head は次のように書ける。

```
safeHead :: [a] -> Maybe a
safeHead []    = Nothing
safeHead (x:_) = Just x
```

通常の head は空リストに対して実行時エラーを起こす。

```
head [] -- runtime error
```

一方、safeHead は失敗可能性を型として表している。

```
safeHead [1, 2, 3] :: Maybe Int
safeHead []       :: Maybe a
```

結果を使う側は、Nothing の場合を処理する必要がある。

```
describeHead :: [Int] -> String
describeHead xs =
  case safeHead xs of
    Nothing -> "empty list"
    Just n   -> "first element is " ++ show n
```

11.2 Maybe の構造

数学的には、

$$\text{Maybe } a \simeq 1 + a$$

のように考えられる。

- Nothing は値がない場合を表す。
- Just a は値がある場合を表す。

12 Either

12.1 理由つきの分岐

Either は次のように定義できる。

```
data Either e a
  = Left e
  | Right a
```

値コンストラクタの型は次のようになる。

```
Left  :: e -> Either e a
Right :: a -> Either e a
```

慣習として、

- Left は失敗・エラー
- Right は成功

を表すことが多い。

```
safeDiv :: Int -> Int -> Either String Int
safeDiv _ 0 = Left "division by zero"
safeDiv x y = Right (x `div` y)
```

ここで、

- 割り算に失敗した場合は Left String
- 割り算に成功した場合は Right Int

を返す。

Maybe と比べると、Either は失敗理由を保持できる。

要点

Maybe a は「成功したか失敗したか」だけを表す。Either e a は「失敗した場合の理由 e」と「成功した場合の値 a」を表す。

13 リスト型

Haskell のリストは特別な構文を持つが、本質的には代数的データ型として理解できる。
概念的には次のような型である。

```
data List a
  = Nil
  | Cons a (List a)
```

Haskell の実際のリストでは、

```
[] :: [a]
(:) :: a -> [a] -> [a]
```

である。

```
[1, 2, 3]
```

は、実際には次と同じである。

```
1 : 2 : 3 : []
```

さらに明示的に書けば、

```
(:) 1 ((:) 2 ((:) 3 []))
```

である。

パターンマッチでは、次のように分解できる。

```
sumList :: [Int] -> Int
sumList []      = 0
sumList (x:xs) = x + sumList xs
```

ここで、

- [] は空リストの場合
- x:xs は先頭要素 x と残りのリスト xs

を表す。

14 レコード型

14.1 通常のデータ型の弱点

次の型を考える。

```
data Person = Person String Int
    deriving (Eq, Show)
```

この型では、String と Int が何を意味するかが、定義だけでは少しわかりにくい。

```
Person "Taro" 20
```

なら、何となく名前と年齢だとわかる。

しかし、

```
Person "Taro" 172
```

だと、172 が年齢なのか、身長なのか、ID なのか曖昧である。

14.2 レコード構文

このような場合、レコード構文を使うと意味が明確になる。

```
data Person = Person
    { name :: String
    , age  :: Int
    }
    deriving (Eq, Show)
```

これは、本質的には String と Int を持つ Person 型を定義しているが、それぞれのフィールドに名前がついている。

このとき、自動的に次のアクセサ関数が作られる。

```
name :: Person -> String
age  :: Person -> Int
```

14.3 レコード型の値を作る

レコード型の値は、通常のコンストラクタ適用でも作れる。

```
taro :: Person
taro = Person "Taro" 20
```

しかし、レコード構文を使うなら、次のように書ける。

```
taro :: Person
taro = Person
  { name = "Taro"
  , age  = 20
  }
```

この書き方では、フィールドの意味が明示される。

また、フィールド名を指定するため、順序に強く依存しない。

```
taro :: Person
taro = Person
  { age  = 20
  , name = "Taro"
  }
```

14.4 レコード型から値を取り出す

レコード型では、フィールド名がアクセサ関数になる。

```
taroName :: String
taroName = name taro

taroAge :: Int
taroAge = age taro
```

アクセサは普通の関数なので、次のようにも使える。

```
people :: [Person]
people =
  [ Person { name = "Taro", age = 20 }
  , Person { name = "Hanako", age = 22 }
  ]

names :: [String]
names = map name people
```

14.5 レコード型とパターンマッチ

レコード型も通常の代数的データ型なので、パターンマッチできる。


```
describePerson :: Person -> String
describePerson (Person n a) =
  n ++ " is " ++ show a ++ " years old."
```

レコード構文を使ってパターンマッチすることもできる。

```
describePerson :: Person -> String
describePerson Person { name = n, age = a } =
  n ++ " is " ++ show a ++ " years old."
```

必要なフィールドだけ取り出すこともできる。

```
personNameLabel :: Person -> String
personNameLabel Person { name = n } =
  "Name: " ++ n
```

この関数では age を使わないため、name だけを取り出している。

14.6 レコード更新構文

Haskell の値は基本的に不変である。したがって、レコードの一部を変更するとは、既存の値を書き換えることなく、一部だけ異なる新しい値を作ることである。

```
birthday :: Person -> Person
birthday p =
  p { age = age p + 1 }
```

これは、

p とほぼ同じだが、age だけを 1 増やした新しい Person を返す

という意味である。

```
olderTaro :: Person
olderTaro = taro { age = 21 }
```

このとき、元の taro が破壊的に変更されるわけではない。

要点

Haskell のレコード更新は、破壊的代入ではなく、新しい値の構築である。

14.7 レコード型は名前付き直積型

次の定義は、

```
data Person = Person
  { name :: String
  , age  :: Int
```

```
}
```

数学的には、おおよそ

$$\text{Person} \simeq \text{String} \times \text{Int}$$

である。

ただし、単なるタプル

```
(String, Int)
```

とは異なり、Person という型名と、name, age というフィールド名によって意味が明確になる。

要点

レコード型は、名前付きの直積型である。

15 レコード型の注意点

15.1 フィールド名は関数名でもある

次のように定義すると、

```
data Person = Person
  { name :: String
  , age  :: Int
  }
```

name はトップレベルの関数として定義される。

```
name :: Person -> String
```

そのため、古典的な Haskell では、同じモジュール内で別の型にも同じフィールド名を使うと衝突することがある。

```
data Person = Person
  { name :: String
  }

data Company = Company
  { name :: String
  }
```

GHC の拡張を使えば同名フィールドを扱いやすくなるが、教育用コードでは、次のように接頭辞を付ける流儀もわかりやすい。

```
data Person = Person
  { personName :: String
  , personAge  :: Int
  }
```

```
}
```

```
data Company = Company
  { companyName :: String
  }
```

15.2 部分的なアクセサに注意

複数のコンストラクタを持つ型でレコード構文を使うと、アクセサが部分関数になる場合がある。

```
data Shape
  = Circle
    { radius :: Double }
  | Rectangle
    { width  :: Double
    , height :: Double
    }
```

このとき、

```
radius :: Shape -> Double
```

というアクセサが作られる。

しかし、

```
radius (Rectangle 3 4)
```

は実行時エラーになる。

なぜなら、Rectangle には radius フィールドがないからである。

安全に処理するには、パターンマッチを使う。

```
area :: Shape -> Double
area Circle { radius = r } =
  pi * r * r
area Rectangle { width = w, height = h } =
  w * h
```

要点

複数コンストラクタのレコード型では、フィールドアクセサが常に安全とは限らない。

16 型変数

16.1 型変数とは何か

次の関数を考える。

```
identity :: a -> a
identity x = x
```

ここで出てくる `a` が型変数である。

`a` は値ではなく、型を表す変数である。

任意の型 `a` について、`a` 型の値を受け取り、同じ `a` 型の値を返す

という意味になる。

具体的には、

```
identity 3      :: Int
identity True   :: Bool
identity "hello" :: String
```

のように使える。

16.2 型変数は使われる場所で具体化される

```
identity :: a -> a
```

において、`a` は任意の型でよい。

ただし、入力と出力は同じ型でなければならない。

- `identity 3` では `a` は `Int` になる。
- `identity True` では `a` は `Bool` になる。
- `identity "hello"` では `a` は `String` になる。

17 多相型

17.1 多相型とは何か

型変数を含む型を多相型という。

```
identity :: a -> a
```

は多相型である。

ひとつの関数が複数の具体型に対して使えるからである。

別の例として、`const` を考える。

```
const :: a -> b -> a
const x y = x
```

これは、

`a` 型の値を受け取り、次に `b` 型の値を受け取り、最初の `a` 型の値を返す

という意味である。

```
const 10 True
const "hello" 3.14
```

のように、異なる型の組み合わせに対して使える。

17.2 型変数の名前

型変数の名前そのものに特別な意味はない。

```
identity :: a -> a
identity x = x
```

と

```
identity :: t -> t
identity x = x
```

は本質的に同じである。

ただし、慣習的には次のような名前がよく使われる。

型変数	よくある意味
a, b, c	一般的な型
e	エラー型、環境型
m	Monad などの文脈を持つ型コンストラクタ
f	Functor などの構造を表す型コンストラクタ
k, v	キーと値
r	結果型、戻り値型、環境型

例えば、

```
lookup :: Ord k => k -> Map k v -> Maybe v
```

では、k は key、v は value と読むと自然である。

18 型変数を持つデータ型

型変数は関数だけでなく、データ型にも使える。

```
data Box a = Box
  { content :: a
  }
deriving (Eq, Show)
```

これは、

任意の型 a の値を 1 つ入れられる箱

である。

```
Box 3      :: Box Int
Box "hello" :: Box String
Box True   :: Box Bool
```

アクセサの型は次のようになる。

```
content :: Box a -> a
```

つまり、

Box a から中身の a を取り出す

という意味である。

19 制約なし多相と制約付き多相

19.1 制約なし多相

次の型を考える。

```
id :: a -> a
```

これは、a に何の制約もない。

したがって、本当に任意の型に対して使える。

```
length :: [a] -> Int
```

も制約なし多相である。

length はリストの要素の中身を見ない。要素が Int でも、String でも、関数でもかまわない。

```
length [1, 2, 3]
length ["a", "b"]
length [True, False]
```

すべて可能である。

19.2 制約付き多相

一方、ある操作を使うには、その操作をサポートする型であることが必要になる。

```
addOne :: Num a => a -> a
addOne x = x + 1
```

ここで、

```
Num a =>
```

は型クラス制約である。

これは、

a が Num のインスタンスであるなら

という意味である。

他にも、次のような制約がよく使われる。

```
show :: Show a => a -> String
(==) :: Eq a => a -> a -> Bool
sort :: Ord a => [a] -> [a]
```

要点

型クラス制約は、多相性を保ちながら、その型に必要な操作能力を要求する仕組みである。

20 多相型の重要な性質

20.1 多相性は自由度を制限する

多相型の重要な点は、

どんな型にも使える関数は、逆に、型固有のことができない

ということである。

例えば、

```
identity :: a -> a
identity x = x
```

では、a が何であるか一切わからない。

したがって、次のようには書けない。

```
bad :: a -> a
bad x = x + 1
```

なぜなら、a が Bool かもしれないからである。

+ を使いたいなら、次のように Num 制約が必要である。

```
addOne :: Num a => a -> a
addOne x = x + 1
```

20.2 型から実装を推測できる

次の型を考える。

```
f :: a -> a
```

この関数は、基本的には受け取った値をそのまま返すしかない。

```
f x = x
```

なぜなら、a 型の値を新しく作る方法がないからである。

別の例として、

```
g :: a -> b -> a
```

を考える。

この関数は、ほぼ次のように振る舞うしかない。

```
g x y = x
```

返り値は a 型であり、手元にある a 型の値は最初の引数 x だけだからである。

要点

Haskell では、型が実装の自由度を制限し、同時にプログラムの意味を説明する。

21 同じ型変数が複数回出る意味

型注釈の中で同じ型変数が複数回出る場合、それらは同じ型でなければならない。

```
pairSame :: a -> a -> (a, a)
pairSame x y = (x, y)
```

この関数では、2 つの引数は同じ型である。

```
pairSame 1 2
pairSame True False
```

は可能である。

しかし、

```
pairSame 1 True
```

はできない。

1 つ目の引数によって a が Int になり、2 つ目の引数によって a が Bool になってしまうからである。

一方、次の関数は異なる型の引数を取れる。

```
pairAny :: a -> b -> (a, b)
pairAny x y = (x, y)
```

この場合、

```
pairAny 1 True
```

が可能である。

要点

$a \rightarrow a \rightarrow (a, a)$ は「同じ型の 2 つ」を表す。 $a \rightarrow b \rightarrow (a, b)$ は「異なる型でもよい 2 つ」を表す。

22 レコード型と多相型の組み合わせ

22.1 多相的な状態型

レコード型と型変数を組み合わせると、抽象度の高いデータ構造を定義できる。

例えば、探索アルゴリズムの状態を次のように表せる。

```
data SearchState node cost = SearchState
  { current    :: node
  , visited    :: [node]
  , totalCost  :: cost
  }
deriving (Eq, Show)
```

この型は、ノードの型 `node` とコストの型 `cost` を固定していない。

例えば、

```
state1 :: SearchState String Int
state1 = SearchState
  { current = "A"
  , visited = ["A", "B"]
  , totalCost = 10
  }
```

では、

```
node ~ String
cost ~ Int
```

である。

一方、

```
state2 :: SearchState Int Double
state2 = SearchState
  { current = 3
  , visited = [1, 2, 3]
  , totalCost = 4.5
  }
```

では、

```
node ~ Int
cost ~ Double
```

である。

22.2 型変数は設計上の自由度を表す

具体的すぎる定義は次のようになる。

```
data IntGraphState = IntGraphState
  { currentNode :: Int
  , visitedNode :: [Int]
  , distance    :: Double
  }
```

これは、ノードが Int、距離が Double であることを固定している。

一方、多相的に書くと次のようになる。

```
data GraphState node dist = GraphState
  { currentNode :: node
  , visitedNode :: [node]
  , distance    :: dist
  }
```

この場合、ノード型や距離型を後から選べる。

```
GraphState Int Double
GraphState String Int
GraphState Vertex Distance
```

要点

型変数は、設計上まだ具体型に固定したくない部分を明示する。

23 type と newtype

23.1 type は別名

type は型の別名を定義する。

```
type Vertex = Int
type Distance = Double
```

この場合、Vertex と Int は型として区別されない。

```
type Vertex = Int
```

は、単に Int に別名を付けているだけである。

23.2 newtype は新しい型を作る

型として区別したい場合は newtype を使う。

```
newtype Vertex = Vertex Int
    deriving (Eq, Ord, Show)

newtype Distance = Distance Double
    deriving (Eq, Ord, Show)
```

これにより、内部表現は `Int` や `Double` でも、型としては別物になる。

要点

`type` は別名を作る。`newtype` は新しい型を作る。

24 型クラス制約とレコード型

多相的なデータ型を使う関数では、必要な操作に応じて型クラス制約を付ける。

例えば、集合 `Set` にノードを入れるには、ノードに順序が必要である。

```
insertVisited :: Ord node
              => node
              -> Set node
              -> Set node
insertVisited n visited =
    Set.insert n visited
```

状態型に対して使うなら、次のように書ける。

```
data GraphState node dist = GraphState
    { currentNode :: node
    , visited      :: Set node
    , distance     :: dist
    }
```

```
markVisited :: Ord node
            => GraphState node dist
            -> GraphState node dist
markVisited st =
    st { visited = Set.insert (currentNode st) (visited st) }
```

ここで `dist` には制約がない。

なぜなら、この関数では距離を見ていないからである。

要点

型シグネチャは、その関数が必要とし、何を必要としていないかを表す。

25 型が設計を表す

Haskell では、型定義を見るだけで設計のかなりの部分がわかる。

例えば、次の型を考える。

```
data LoginResult
  = LoginSuccess User
  | InvalidPassword
  | UserNotFound
  | AccountLocked
```

この型を見るだけで、ログイン処理の結果として何が起こりうるかがわかる。

```
login :: UserName -> Password -> IO LoginResult
```

という型があれば、

ユーザー名とパスワードを受け取り、IO を伴ってログイン結果を返す

ということがわかる。

結果を処理する側は、すべての場合を考える必要がある。

```
handleLogin :: LoginResult -> String
handleLogin result =
  case result of
    LoginSuccess user -> "Welcome"
    InvalidPassword    -> "Invalid password"
    UserNotFound       -> "User not found"
    AccountLocked      -> "Account locked"
```

要点

Haskell の型定義は、実装であると同時に設計文書でもある。

26 説明順序の提案

人に説明する場合は、次の順序がわかりやすい。

1. Bool を使って、型と値コンストラクタの区別を説明する。
2. Point を使って、値コンストラクタが関数であることを説明する。
3. Shape を使って、直和型とパターンマッチを説明する。
4. Maybe を使って、失敗可能性を型で表すことを説明する。
5. Either を使って、理由付きの成功・失敗を説明する。
6. レコード型を使って、名前付き直積型を説明する。
7. 型変数と多相型を使って、抽象度の高い設計を説明する。

この流れにすると、

Haskell の型は、単なるデータ分類ではなく、プログラムの意味構造を表す

という理解へ自然に導ける。

27 まとめ

Haskell の型システムは、値がどのような構造を持ち、どのような場合分けを持ち、どのような計算として扱われるべきかを静的に記述する仕組みである。

特に、代数的データ型では、

```
data T
  = C1 ...
  | C2 ...
  | C3 ...
```

という定義そのものが、

T 型の値は C1 の形、C2 の形、または C3 の形のいずれかである

という仕様になっている。

また、レコード型は値の内部構造に名前を与える仕組みであり、型変数はまだ具体型に固定しない部分を明示する仕組みである。

したがって、両者を組み合わせると、データの意味と抽象度を同時に型定義の中に表現できる。

要点

Haskell において型は、コンパイラ向けの注釈ではなく、設計そのものを記述する言語である。